

ADP470S – Applications Development Practice 4

Test 1 – Comprehensive Answer Guide

QUESTION 1 – Big O Expression for Code Excerpts

1.1 – Two Sequential Loops

```
int sum = 0;
for (int i = 0; i < n; i++) {           // Loop A: runs n times → O(n)
    sum = sum + a[i];                   // O(1) work per iteration
}
for (int j = 0; j < n; j++) {           // Loop B: runs n times → O(n)
    sum = sum + a[j];                   // O(1) work per iteration
}
```

Analysis:

Loop A executes n times $\rightarrow O(n)$.

Loop B also executes n times $\rightarrow O(n)$.

The two loops are sequential (not nested), so we ADD their complexities:

$$T(n) = O(n) + O(n) = O(2n)$$

By the constant rule, we drop the constant multiplier:

Big O = $O(n)$

1.2 – Nested Loops (Outer linear, Inner multiplicative)

```
int prod = 1;
for (int i = 1; i < n; i++) {           // Outer loop: i goes 1,2,...,n-1 → runs
n-1 times → O(n)
    for (int j = i; j < n; j = j * 3) { // Inner loop: j starts at i, triples
each step
        prod = prod * j;                 // j = i, 3i, 9i, 27i, ... until j >=
n
    }                                     // Number of inner iterations ≈ log3(n/i)
}
```

Analysis:

The outer loop runs $O(n)$ times.

The inner loop: j starts at i and multiplies by 3 each time until $j \geq n$.

Number of inner iterations = $\lfloor \log_3(n/i) \rfloor \approx O(\log n)$ per outer iteration.

Since each outer iteration does $O(\log n)$ inner iterations:

$$T(n) = O(n) \times O(\log n)$$

Big O = $O(n \log n)$

1.3 – Nested Loops (Outer stride 3, Inner linear)

```
int sum = 0;
for (i = 0; i <= n-1; i += 3) {         // Outer loop: increments by 3 each time
// i = 0, 3, 6, 9, ..., ≤ n-1
```

```

// Runs approximately n/3 times →  $O(n/3) =$ 
 $O(n)$ 
for (j = n-1; j > 0; j--) { // Inner loop: j goes from n-1 down to 1 →
 $O(n)$ 
    sum = sum + (i * j); //  $O(1)$  work
}
}

```

Analysis:

Outer loop: i increments by 3 each pass → runs $\approx n/3$ times. $O(n/3) = O(n)$ by the constant rule.

Inner loop: j decrements from n-1 to 1 → runs $\approx n$ times → $O(n)$.

The loops are NESTED, so we MULTIPLY:

$$T(n) = O(n) \times O(n) = O(n^2/3)$$

Big O = $O(n^2)$

QUESTION 2 – Recursive Factorial: Code, Stack Frames & Complexity

2.1 – Recursive Factorial Implementation (Java)

```
public class Factorial {  
  
    /**  
     * Recursive factorial method.  
     * Base case: factorial(0) = 1, factorial(1) = 1  
     * Recursive case: factorial(n) = n * factorial(n-1)  
     *  
     * @param n the non-negative integer to compute factorial for  
     * @return n!  
     */  
    public static long factorialRecursive(int n) {  
        // Base case: 0! = 1 and 1! = 1 – stop recursing  
        if (n == 0 || n == 1) {  
            return 1;  
        }  
        // Recursive case: n! = n * (n-1)!  
        // The method calls itself with a SMALLER input, building a call stack  
        return n * factorialRecursive(n - 1);  
    }  
  
    public static void main(String[] args) {  
        int n = 5;  
        System.out.println(n + "! = " + factorialRecursive(n)); // Output: 5! =  
120  
    }  
}
```

2.1 – Stack Frame Trace for factorial(5)

Each time the recursive method is called, a new STACK FRAME is pushed onto the call stack. The frame stores the local variable *n* and the return address.

CALL STACK (grows downward as each call is made):

factorialRecursive(5)	→ waits for result	← TOP of stack initially
factorialRecursive(4)	→ waits for result	
factorialRecursive(3)	→ waits for result	
factorialRecursive(2)	→ waits for result	
factorialRecursive(1)	→ returns 1 (BASE CASE)	← Bottom

UNWINDING (stack frames popped as results propagate back up):

```
factorialRecursive(1) = 1  
factorialRecursive(2) = 2 * 1 = 2  
factorialRecursive(3) = 3 * 2 = 6  
factorialRecursive(4) = 4 * 6 = 24  
factorialRecursive(5) = 5 * 24 = 120
```

2.2 – Time & Space Complexity (Recursive Factorial)

Time Complexity: $O(n)$

The function makes exactly *n* recursive calls before hitting the base case.

Each call does $O(1)$ work (one multiplication).

Total work = $n \times O(1) = O(n)$.

Space Complexity: $O(n)$

Each recursive call creates a NEW stack frame on the call stack.

For factorial(n), the stack grows to a depth of n frames simultaneously.

Therefore, memory usage is proportional to $n \rightarrow O(n)$ space.

This is the key DISADVANTAGE of recursion vs. iteration.

Iterative Factorial (for comparison)

```
/**
 * Iterative factorial – uses a loop instead of recursion.
 * Avoids building up a call stack, so space complexity =  $O(1)$ .
 */
public static long factorialIterative(int n) {
    long result = 1; // Accumulator, starts at 1 (identity for multiplication)

    // Loop from 2 up to n, multiplying result by each integer
    for (int i = 2; i <= n; i++) {
        result = result * i; // e.g. for n=5: 1*2=2, *3=6, *4=24, *5=120
    }
    return result; // Returns n!
}
```

Iterative Complexity:

Time: $O(n)$ — loop runs $n-1$ times.

Space: $O(1)$ — only the variable 'result' is needed, no call stack growth.

Conclusion – Which is preferable?

Both have $O(n)$ time complexity. However, the ITERATIVE approach is preferred in practice because:

1. Space: $O(1)$ vs $O(n)$ — iterative does not consume stack space.
2. Stack overflow risk: deep recursion (large n) can exceed the JVM call stack limit and throw a `StackOverflowError`.
3. Performance: function call overhead per recursive call adds constant factors.

QUESTION 3 – Sorting Algorithms on {90, 23, 5, 109, 12}

3.1 – Bubble Sort: Step-by-Step

Starting array: { 90, 23, 5, 109, 12 }

Bubble Sort repeatedly compares adjacent elements and swaps them if they are out of order. After each pass, the largest unsorted element 'bubbles' to its correct position at the end.

```
Initial: { 90, 23, 5, 109, 12 }

— PASS 1 (compare n-1 = 4 pairs) —————
Compare a[0]=90 vs a[1]=23 → 90 > 23 → SWAP { 23, 90, 5, 109, 12 }
Compare a[1]=90 vs a[2]=5 → 90 > 5 → SWAP { 23, 5, 90, 109, 12 }
Compare a[2]=90 vs a[3]=109 → 90 < 109 → no swap { 23, 5, 90, 109, 12 }
Compare a[3]=109 vs a[4]=12 → 109 > 12 → SWAP { 23, 5, 90, 12, 109 }
End of Pass 1: 109 is sorted at position 4 ✓

— PASS 2 (compare 3 pairs) —————
Compare a[0]=23 vs a[1]=5 → 23 > 5 → SWAP { 5, 23, 90, 12, 109 }
Compare a[1]=23 vs a[2]=90 → 23 < 90 → no swap { 5, 23, 90, 12, 109 }
Compare a[2]=90 vs a[3]=12 → 90 > 12 → SWAP { 5, 23, 12, 90, 109 }
End of Pass 2: 90 is sorted at position 3 ✓

— PASS 3 (compare 2 pairs) —————
Compare a[0]=5 vs a[1]=23 → 5 < 23 → no swap { 5, 23, 12, 90, 109 }
Compare a[1]=23 vs a[2]=12 → 23 > 12 → SWAP { 5, 12, 23, 90, 109 }
End of Pass 3: 23 is sorted at position 2 ✓

— PASS 4 (compare 1 pair) —————
Compare a[0]=5 vs a[1]=12 → 5 < 12 → no swap { 5, 12, 23, 90, 109 }
End of Pass 4: All elements sorted ✓

Final sorted array: { 5, 12, 23, 90, 109 }
```

3.2 – Bubble Sort: Big O Expression

```
/**
 * Bubble Sort Implementation
 *
 * Time Complexity:
 * Best Case: O(n) - array already sorted; optimised version detects no
swaps
 * Average Case: O(n2) - roughly half of all comparisons result in swaps
 * Worst Case: O(n2) - array in reverse order; every comparison is a swap
 *
 * Space Complexity: O(1) - sorting is done in-place with only a temp variable
 */
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    boolean swapped;

    // Outer loop: each pass guarantees one more element is in final position
    for (int pass = 0; pass < n - 1; pass++) {
        swapped = false; // Optimisation: detect if array is already sorted

        // Inner loop: compare adjacent pairs up to the last unsorted position
        // After each pass, the largest element has moved to the end, so we
        // reduce the inner loop range by 'pass'
        for (int j = 0; j < n - 1 - pass; j++) {
            if (arr[j] > arr[j + 1]) { // Out of order? Then swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
    }
}
```

```
        arr[j + 1] = temp;
        swapped = true;           // A swap occurred this pass
    }
    // If no swaps occurred, array is already sorted - exit early (best
case O(n))
    if (!swapped) break;
}
}
```

Big O Explanation:

The outer loop runs at most $n-1$ times.

The inner loop runs $(n-1)$, $(n-2)$, ..., 1 times across all passes.

Total comparisons = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2 \approx n^2/2 \rightarrow O(n^2)$.

3.3 – Quick Sort: Step-by-Step on {90, 23, 5, 109, 12}

Quick Sort selects a PIVOT element, partitions the array into elements less than the pivot (left) and greater than the pivot (right), then recursively sorts each partition. Pivot = MIDDLE element of current subarray.

```
Initial: { 90, 23, 5, 109, 12 }

— LEVEL 1: Full array [0..4] —————
Pivot index = (0+4)/2 = 2 → pivot = arr[2] = 5
Partition:
  Elements < 5 : { }           (nothing less than 5)
  Element = 5  : { 5 }
  Elements > 5 : { 90, 23, 109, 12 }
Result after partition: { 5 | 90, 23, 109, 12 }
→ 5 is in its correct position (index 0) ✓

— LEVEL 2: Right subarray { 90, 23, 109, 12 } —————
Pivot index = (0+3)/2 = 1 → pivot = arr[1] = 23
Partition:
  Elements < 23 : { 12 }
  Element = 23  : { 23 }
  Elements > 23 : { 90, 109 }
Result: { 5, 12, 23, 90, 109 }
→ 23 is in its correct position (index 2) ✓

— LEVEL 3a: Left sub-subarray { 12 } —————
Single element → already sorted ✓

— LEVEL 3b: Right sub-subarray { 90, 109 } —————
Pivot index = (0+1)/2 = 0 → pivot = arr[0] = 90
Partition:
  Elements < 90 : { }
  Element = 90  : { 90 }
  Elements > 90 : { 109 }
→ 90 and 109 are in correct positions ✓

Final sorted array: { 5, 12, 23, 90, 109 }
```

3.4 – Quick Sort: Big O Expression

```
/**
 * Quick Sort Implementation
 *
 * Time Complexity:
 *   Best Case:    O(n log n)  – pivot always splits array into equal halves
 *   Average Case: O(n log n)  – expected behaviour with random data
 *   Worst Case:   O(n2)       – pivot is always the smallest/largest element
 *                               (e.g. sorted array with first/last as pivot)
 *
 * Space Complexity:
 *   O(log n) average – recursive call stack depth
 *   O(n)      worst case – degenerate (skewed) partitioning
 */
public static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        // Choose middle element as pivot to reduce worst-case probability
        int mid = low + (high - low) / 2;
        int pivot = arr[mid];

        // Move pivot to end temporarily
        swap(arr, mid, high);

        // Partition: place all elements smaller than pivot to the left
        int i = low - 1; // i tracks the boundary of the 'less than' region
        for (int j = low; j < high; j++) {
```

```

        if (arr[j] <= pivot) { // Current element belongs in the left
partition
            i++;
            swap(arr, i, j); // Move it to the left side
        }
    }
    // Place pivot in its final sorted position
    swap(arr, i + 1, high);
    int pivotIndex = i + 1;

    // Recursively sort the left and right partitions
    quickSort(arr, low, pivotIndex - 1); // Elements < pivot
    quickSort(arr, pivotIndex + 1, high); // Elements > pivot
}

private static void swap(int[] arr, int a, int b) {
    int temp = arr[a];
    arr[a] = arr[b];
    arr[b] = temp;
}

```

Big O Explanation:

Best/Average Case $O(n \log n)$: The array is divided into two halves at each level ($\log n$ levels), and partitioning each level takes $O(n)$ total work.

Worst Case $O(n^2)$: If the pivot always lands at the extreme (e.g., smallest or largest), partitions are 0 and $n-1$, creating n levels instead of $\log n$.

3.5 – Heap Sort: Step-by-Step on {90, 23, 5, 109, 12}

Heap Sort works in two phases:

Phase 1 – Build a MAX-HEAP from the array (largest element at root/index 0).

Phase 2 – Repeatedly extract the maximum (swap root with last unsorted element, then re-heapify).

```
Initial array: { 90, 23, 5, 109, 12 }
                Index: 0  1  2  3  4
```

Parent-child relationships in heap array:

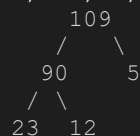
```
Parent of i    = (i-1)/2
Left child  of i = 2i+1
Right child of i = 2i+2
```

```
===== PHASE 1: BUILD MAX-HEAP =====
Start heapifying from last non-leaf: index = n/2 - 1 = 1
```

```
Heapify at index 1 (value=23):
Left child  = index 3 → 109
Right child = index 4 → 12
Largest child: 109 (index 3) > 23 → SWAP index 1 and 3
Array: { 90, 109, 5, 23, 12 }
```

```
Heapify at index 0 (value=90):
Left child  = index 1 → 109
Right child = index 2 → 5
Largest child: 109 (index 1) > 90 → SWAP index 0 and 1
Array: { 109, 90, 5, 23, 12 }
(Index 1 has no children needing adjustment)
```

Max-Heap built: { 109, 90, 5, 23, 12 }



```
===== PHASE 2: EXTRACT MAX REPEATEDLY =====
```

```
— Iteration 1 (heap size = 5) —————
Swap root (109) with last element (12):
{ 12, 90, 5, 23, | 109 } ← 109 is now sorted ✓
Heapify { 12, 90, 5, 23 } at index 0:
Left=90, Right=5 → 90 is largest → SWAP 0 and 1
{ 90, 12, 5, 23, | 109 }
Heapify at index 1 (value=12): Left child=23 → SWAP 1 and 3
{ 90, 23, 5, 12, | 109 }
```

```
— Iteration 2 (heap size = 4) —————
Swap root (90) with last unsorted (12):
{ 12, 23, 5, | 90, 109 } ← 90 sorted ✓
Heapify { 12, 23, 5 } at index 0:
Left=23, Right=5 → 23 is largest → SWAP 0 and 1
{ 23, 12, 5, | 90, 109 }
```

```
— Iteration 3 (heap size = 3) —————
Swap root (23) with last unsorted (5):
{ 5, 12, | 23, 90, 109 } ← 23 sorted ✓
Heapify { 5, 12 } at index 0:
Left=12 (no right child) → 12 > 5 → SWAP 0 and 1
{ 12, 5, | 23, 90, 109 }
```

```
— Iteration 4 (heap size = 2) —————
Swap root (12) with last unsorted (5):
{ 5, | 12, 23, 90, 109 } ← 12 sorted ✓
Single element left → done
```

```
Final sorted array: { 5, 12, 23, 90, 109 }
```

3.6 – Heap Sort: Big O Expression

```
/**
 * Heap Sort Implementation
 *
 * Time Complexity:
 *   Best Case:    O(n log n)  - building heap is O(n); each of n extractions
 *                           requires a heapify of O(log n)
 *   Average Case: O(n log n)  - same reasoning applies
 *   Worst Case:   O(n log n)  - Heap Sort has NO worst-case degradation to
O(n2)
 *
 *                               unlike Quick Sort; this is its main advantage.
 *
 * Space Complexity: O(1)  - sorting is done in-place; only a temp var needed
 */
public static void heapSort(int[] arr) {
    int n = arr.length;

    // Phase 1: Build max-heap
    // Start from last non-leaf node and heapify each node upward to root
    // Last non-leaf is at index n/2 - 1
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i); // Each heapify is O(log n); total O(n) for phase
1    }

    // Phase 2: Extract elements from heap one by one
    for (int i = n - 1; i > 0; i--) {
        // Move current root (maximum) to end of the array
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        // Restore the heap property for the reduced heap (size = i)
        heapify(arr, i, 0); // O(log i) per call → total O(n log n) for phase
2    }
}

/**
 * Heapify subtree rooted at index 'root' in array of size 'n'.
 * Ensures the subtree satisfies the max-heap property.
 */
private static void heapify(int[] arr, int n, int root) {
    int largest = root; // Assume root is the largest
    int left  = 2 * root + 1; // Left child index
    int right = 2 * root + 2; // Right child index

    // Check if left child exists and is greater than root
    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    // Check if right child exists and is greater than current largest
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    // If root is NOT the largest, swap and recursively heapify the affected
    subtree
    if (largest != root) {
        int temp = arr[root];
        arr[root] = arr[largest];
        arr[largest] = temp;
        heapify(arr, n, largest); // Recursively fix the swapped subtree
    }
}
```

```
}
```

Big O Explanation:

Phase 1 (Build Heap): Although it appears to be $O(n \log n)$, mathematical analysis shows building a heap from an array takes only $O(n)$.

Phase 2 (Extract): Extracting n elements, each requiring a heapify of $O(\log n) \rightarrow O(n \log n)$ total.

Combined: $O(n) + O(n \log n) = O(n \log n)$ for all cases.

Heap Sort guarantees $O(n \log n)$ in ALL cases, making it more reliable than Quick Sort for adversarial inputs.